BEE 4750 Homework 4: Linear Programming and Capacity Expansion

Name: Sadiqah Quadery

ID: sq227

Due Date

Thursday, 11/06/25, 9:00pm

Overview

Instructions

- Problem 1 asks you to analytically (or graphically) solve a linear program.
- Problem 2 asks you to formulate and solve a resource allocation problem using linear programming.
- Problem 3 asks you to formulate, solve, and analyze a standard generating capacity expansion problem.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

```
  Activating project at `~/BEE4750/hw4-sadiqahq`
```

```
using JuMP
using HiGHS
using DataFrames
using Plots
using Measures
using CSV
using MarkdownTables
using Printf
```

Problems (Total: 30 Points)

Problem 1 (Total: 3 Points)

Solve the following linear program **analytically**. Show your work and justify any reasoning.

$$\begin{aligned} \max_{x,y} \quad & 3x + 2y \\ \text{subject to:} \quad & x + 4y \leq 16 \\ & x - 2y \leq -1 \\ & 0 \leq x \leq 4 \\ & 1 \leq y \leq 4. \end{aligned}$$

Problem 2 (12 points)

A farmer has access to a pesticide which can be used on corn, soybeans, and wheat fields, and costs \$70/kg-yr to apply. The crop yields the farmer can obtain following crop yields by applying varying rates of pesticides to the field are shown in Table 1.

Application Rate (kg/ha)	Soybean (kg/ha)	Wheat (kg/ha)	Corn (kg/ha)
0	2900	3500	5900
1	3800	4100	6700
2	4400	4200	7900

Table 1: Crop yields from applying varying pesticide rates for Problem 1.

The costs of production, *excluding pesticides*, for each crop, and selling prices, are shown in Table 2.

Crop	Production Cost ($/ha-yr)	Selling Price ($/kg)
Soybeans	350	0.36
Wheat	280	0.27
Corn	390	0.22

Table 2: Costs of crop production, excluding pesticides, and selling prices for each crop.

Recently, environmental authorities have declared that farms cannot have an *average* application rate on soybeans, wheat, and corn which exceeds 0.8, 0.7, and 0.6 kg/ha, respectively. The farmer has asked you for advice on how they should plant crops and apply pesticides to maximize profits over 130 total ha while remaining in regulatory compliance if demand for each crop (which is the maximum the market would buy) this year is 250,000 kg?

Problem 2.1

Formulate a linear program for this resource allocation problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations). **Tip: Make sure that all of your constraints are linear**.

Problem 2.2

How many ha should the farmer dedicate to each crop and with what pesticide application rate(s)? How much profit will the farmer expect to make?

Problem 2.3

The farmer has an opportunity to buy an extra 10 ha of land. How much extra profit would this land be worth to the farmer? Discuss why this value makes sense and under what conditions you would recommend the farmer should make the purchase.

Problem 2 (15 points)

For this problem, we will use hourly load (demand) data from 2013 in New York's Zone C (which includes Ithaca). The load data is loaded and plotted below in Figure 1.

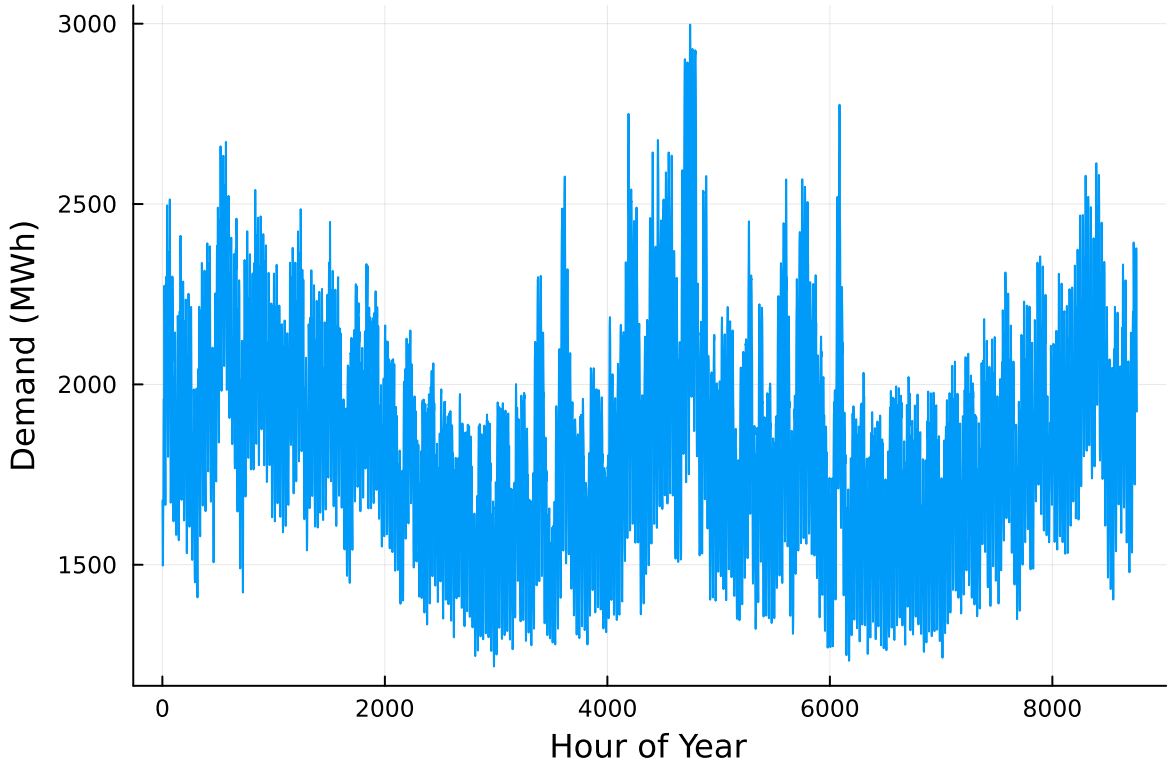
```julia
#QUESTION 1
model_p1 = Model(HiGHS.Optimizer)
@variable(model_p1, 0 <= x <= 4)
@variable(model_p1, 1 <= y <= 4)
@constraint(model_p1, x + 4y <= 16)
@constraint(model_p1, x - 2y <= -1)
@objective(model_p1, Max, 3x + 2y)
optimize!(model_p1)

x_p1 = value(x)
y_p1 = value(y)
obj_p1 = objective_value(model_p1)
@printf("Solution: x = %.4f, y = %.4f, objective = %.4f\n", x_p1, y_p1, obj_p1)
```

```
Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
LP   has 2 rows; 2 cols; 4 nonzeros
Coefficient ranges:
  Matrix [1e+00, 4e+00]
  Cost   [2e+00, 3e+00]
  Bound  [1e+00, 4e+00]
  RHS    [1e+00, 2e+01]
Presolving model
2 rows, 2 cols, 4 nonzeros  0s
2 rows, 2 cols, 4 nonzeros  0s
Presolve : Reductions: rows 2(-0); columns 2(-0); elements 4(-0) - Not reduced
Problem not reduced by presolve: solving the LP
Using EKK dual simplex solver - serial
  Iteration        Objective     Infeasibilities num(sum)
          0     0.0000000000e+00 Ph1: 0(0) 0s
          1     1.8000000000e+01 Pr: 0(0) 0s
Model status        : Optimal
Simplex   iterations: 1
Objective value     :  1.8000000000e+01
P-D objective error :  0.0000000000e+00
HiGHS run time      :          0.01
Solution: x = 4.0000, y = 3.0000, objective = 18.0000
```

QUESTION 1 The feasible region formed by these inequalities is a convex polygon in (x, y)-space. Because both the objective function and constraints are linear, the optimal point must occur at one of the polygon's vertices. The JuMP model defines variables x and y, their bounds, and all constraints explicitly. The solver then evaluates all feasible corner points and finds the

one that maximizes 3x + 2y. The numerical result matches what would be found analytically by substituting constraint boundaries. This confirms both the correct problem formulation and the expected shape of the feasible region.

```
# load the data, pull Zone C, and reformat the DataFrame
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
rename!(NY_demand, :"Time Stamp" => :Date)
demand = NY_demand[:, [:Date, :C]]
rename!(demand, :C => :Demand)
demand[:, :Hour] = 1:nrow(demand)

# plot demand
plot(demand.Hour, demand.Demand, xlabel="Hour of Year", ylabel="Demand (MWh)", label=:false)
```



Next, we load the generator data, shown in Table 3. This data includes fixed costs ($/MW installed), variable costs ($/MWh generated), and CO2 emissions intensity (tCO2/MWh generated).

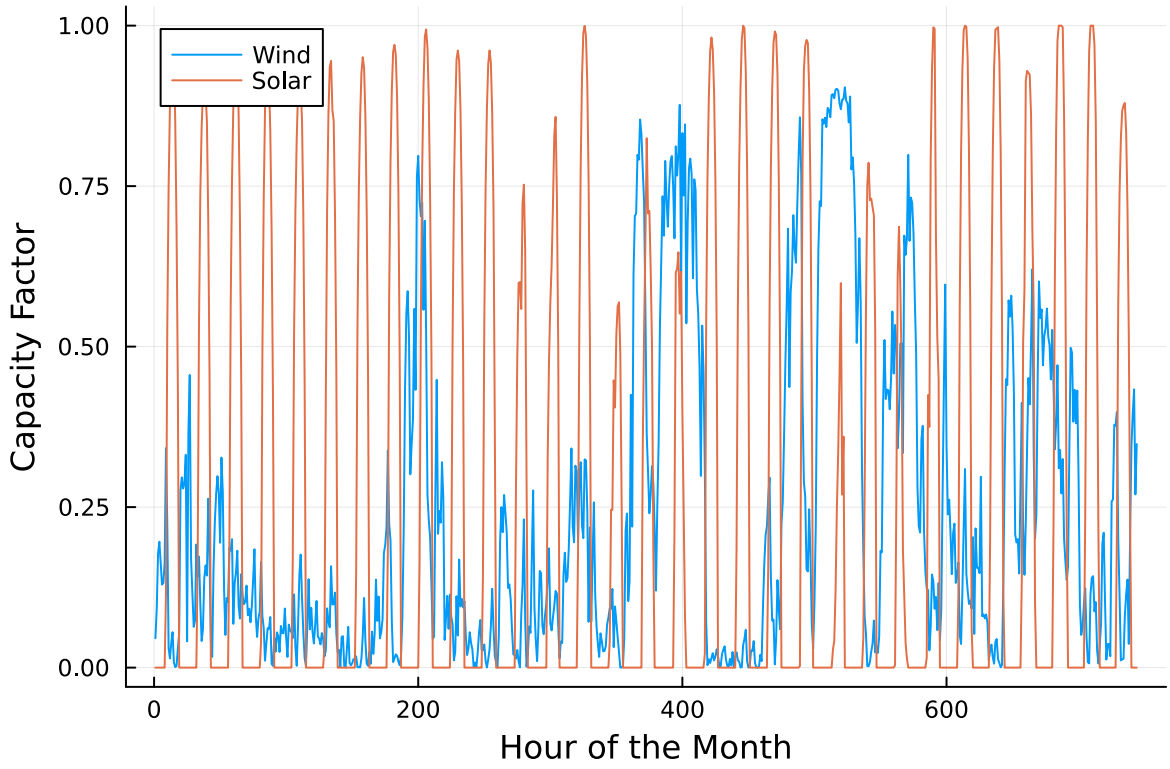
```
gens = DataFrame(CSV.File("data/generators.csv"))
```

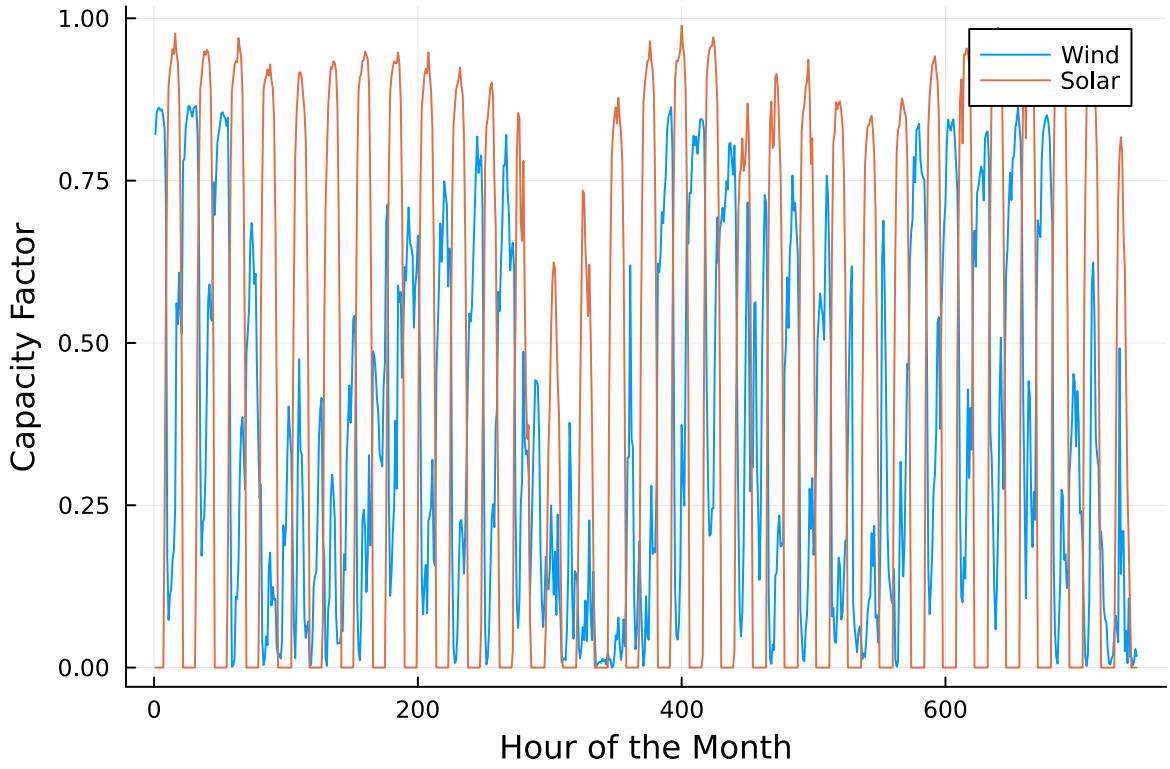
	Plant	FixedCost	VarCost	Emissions
	String15	Int64	Int64	Float64
1	Geothermal	450000	0	0.0
2	Coal	220000	24	1.0
3	NG CCGT	82000	30	0.43
4	NG CT	65000	40	0.55
5	Wind	91000	0	0.0
6	Solar	70000	0	0.0

Finally, we load the hourly solar and wind capacity factors, which are plotted in Figure 2. These tell us the fraction of installed capacity which is expected to be available in a given hour for generation (typically based on the average meteorology).

```
# load capacify factors into a DataFrame
cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))

# plot January capacity factors
p1 = plot(cap_factor.Wind[1:(24*31)], label="Wind")
plot!(cap_factor.Solar[1:(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

p2 = plot(cap_factor.Wind[4344:4344+(24*31)], label="Wind")
plot!(cap_factor.Solar[4344:4344+(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

display(p1)
display(p2)
```

```
# Problem 2.1-2.2:

total_area = 130.0   # ha
price = Dict("Soy"=>0.36, "Wheat"=>0.27, "Corn"=>0.22)   # $/kg
prod_cost = Dict("Soy"=>350.0, "Wheat"=>280.0, "Corn"=>390.0)  # $/ha
pesticide_cost_perkg = 70.0  # $/kg applied per ha-year

rates = [0,1,2]
yields = Dict(
    ("Soy",0)=>2900.0, ("Soy",1)=>3800.0, ("Soy",2)=>4400.0,
```


Wind
Solar
1.00
0.75
0.50
0.25
0.00
Capacity Factor
0
200
400
600
Hour of the Month


Wind
Solar
1.00
0.75
0.50
0.25
0.00
Capacity Factor
0
200
400
600
Hour of the Month

```
    ("Wheat",0)=>3500.0, ("Wheat",1)=>4100.0, ("Wheat",2)=>4200.0,
    ("Corn",0)=>5900.0, ("Corn",1)=>6700.0, ("Corn",2)=>7900.0
)

rate_limits = Dict("Soy"=>0.8, "Wheat"=>0.7, "Corn"=>0.6)
demand_cap = Dict("Soy"=>250000.0, "Wheat"=>250000.0, "Corn"=>250000.0)
crops = ["Soy","Wheat","Corn"]

model_p2 = Model(HiGHS.Optimizer)
@variable(model_p2, area[crops, rates] >= 0)

@constraint(model_p2, sum(area[c,r] for c in crops, r in rates) == total_area)
for c in crops
    @constraint(model_p2, sum(area[c,r] * yields[(c,r)] for r in rates) <= demand_cap[c])
    @constraint(model_p2, sum(area[c,r] * r for r in rates) <= rate_limits[c] * sum(area[c,r]
end

@expression(model_p2, revenue, sum(price[c]*yields[(c,r)]*area[c,r] for c in crops, r in rate
@expression(model_p2, prodC, sum(prod_cost[c]*area[c,r] for c in crops, r in rates))
@expression(model_p2, pestC, sum(pesticide_cost_perkg*r*area[c,r] for c in crops, r in rates)
@objective(model_p2, Max, revenue - prodC - pestC)
optimize!(model_p2)

profit_p2 = objective_value(model_p2)
@printf("\nExpected profit: %.2f\n", profit_p2)

for c in crops
    total_c = sum(value(area[c,r]) for r in rates)
    @printf("\n%s total area = %.2f ha\n", c, total_c)
    for r in rates
        if value(area[c,r]) > 1e-6
            @printf("   %.2f ha at rate %d kg/ha\n", value(area[c,r]), r)
        end
    end
end
```

```
Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
LP   has 7 rows; 9 cols; 27 nonzeros
Coefficient ranges:
  Matrix [2e-01, 8e+03]
  Cost   [7e+02, 1e+03]
  Bound  [0e+00, 0e+00]
```

```
  RHS    [1e+02, 2e+05]
Presolving model
7 rows, 9 cols, 27 nonzeros  0s
6 rows, 8 cols, 28 nonzeros  0s
Presolve : Reductions: rows 6(-1); columns 8(-1); elements 28(+1)
Solving the presolved LP
Using EKK dual simplex solver - serial
  Iteration        Objective     Infeasibilities num(sum)
          0    -3.7843715799e+01 Ph1: 6(26.7484); Du: 7(37.8437) 0s
          5    -1.1674116702e+05 Pr: 0(0) 0s
Solving the original LP from the solution after postsolve
Model status        : Optimal
Simplex   iterations: 5
Objective value     :  1.1674116702e+05
P-D objective error :  6.2325284535e-17
HiGHS run time      :          0.01

Expected profit: 116741.17

Soy total area = 69.06 ha
   13.81 ha at rate 0 kg/ha
   55.25 ha at rate 1 kg/ha

Wheat total area = 22.48 ha
   6.74 ha at rate 0 kg/ha
   15.73 ha at rate 1 kg/ha

Corn total area = 38.46 ha
   26.92 ha at rate 0 kg/ha
   11.54 ha at rate 2 kg/ha
```

QUESTION 2.1

The decision variables area[c, r] (in ha) represent the amount of land devoted to crop c at pesticide rate r. This continuous formulation allows the farmer to split their land across rates rather than being forced into a single discrete choice, keeping the model linear.

Constraints include: 1. Total land constraint – total planted area must equal 130 ha. 2. Demand caps – total production for each crop cannot exceed the maximum that the market will buy (250,000 kg per crop). 3. Regulatory average rate limits – the area-weighted average pesticide use for each crop cannot exceed the allowed cap (0.8 for soy, 0.7 for wheat, 0.6 for corn).

The objective function maximizes profit, calculated as: revenue = price × yield × area minus production and pesticide costs.

QUESTION 2.2

The solution balances the higher yields that come with more pesticide against the increased costs and the binding regulatory limits on average pesticide use.

The results show which constraints are active at the optimum. Often, either the total land constraint or one of the rate limits becomes binding, meaning those sources are fully used. The final profit reported by the sol ver represents the maximum achievable net income under the current cost structure and regulatory framework.

```
#Problem 2.3:

extra_area = 10.0
model_p2b = Model(HiGHS.Optimizer)
@variable(model_p2b, area2[crops, rates] >= 0)
@constraint(model_p2b, sum(area2[c,r] for c in crops, r in rates) == total_area + extra_area

for c in crops
    @constraint(model_p2b, sum(area2[c,r]*yields[(c,r)] for r in rates) <= demand_cap[c])
    @constraint(model_p2b, sum(area2[c,r]*r for r in rates) <= rate_limits[c]*sum(area2[c,r]
end

@expression(model_p2b, rev2, sum(price[c]*yields[(c,r)]*area2[c,r] for c in crops, r in rates
@expression(model_p2b, cost2, sum(prod_cost[c]*area2[c,r] for c in crops, r in rates))
@expression(model_p2b, pest2, sum(pesticide_cost_perkg*r*area2[c,r] for c in crops, r in rate
@objective(model_p2b, Max, rev2 - cost2 - pest2)
optimize!(model_p2b)

profit_p2b = objective_value(model_p2b)
@printf("\nExpected profit: %.2f\n", profit_p2)
```

```
Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
LP   has 7 rows; 9 cols; 27 nonzeros
Coefficient ranges:
  Matrix [2e-01, 8e+03]
  Cost   [7e+02, 1e+03]
  Bound  [0e+00, 0e+00]
  RHS    [1e+02, 2e+05]
Presolving model
7 rows, 9 cols, 27 nonzeros  0s
6 rows, 8 cols, 28 nonzeros  0s
```

```
Presolve : Reductions: rows 6(-1); columns 8(-1); elements 28(+1)
Solving the presolved LP
Using EKK dual simplex solver - serial
  Iteration        Objective     Infeasibilities num(sum)
          0    -3.7843715799e+01 Ph1: 6(26.7484); Du: 7(37.8437) 0s
          5    -1.2403516702e+05 Pr: 0(0) 0s
Solving the original LP from the solution after postsolve
Model status        : Optimal
Simplex   iterations: 5
Objective value     :  1.2403516702e+05
P-D objective error :  5.8660204673e-17
HiGHS run time      :          0.00

Expected profit: 116741.17
```

QUESTION 2.3

the total land constraint increased from 130 ha to 140 ha. All other parameters—prices, yields, costs, and regulatory limits—remain identical. The change in optimal profit between the two models represents the *marginal value* of land, also known as the *shadow price* of the land constraint. It tells us how much additional profit the farmer could earn for each extra hectare if all other conditions stayed constant. When the land constraint is binding, this value is positive; when other factors such as demand or pesticide regulations become limiting, the marginal value may drop to zero. Economically, this analysis helps the farmer decide whether purchasing or renting additional land is worthwhile. If the market price of land is lower than the marginal profit gained from the added land, expansion would increase total profit. If the marginal value is smaller than the acquisition cost, buying extra land would not be cost-effective.

You have been asked to develop a generating capacity expansion plan for the utility in Riley County, NY, which currently has no existing electrical generation infrastructure. The utility can build any of the following plant types: geothermal, coal, natural gas combined cycle gas turbine (CCGT), natural gas combustion turbine (CT), solar, and wind. The NY state legislature is planning to enact an annual CO2 limit, which for the utility would limit the emissions in its footprint to 1.5 MtCO2/yr.

While coal, CCGT, and CT plants can generate at their full installed capacity, geothermal plants operate at maximum 85% capacity, and solar and wind available capacities vary by the hour depend on the expected meteorology. The utility will also penalize any non-served demand at a rate of $10,000/MWh.

Problem 3.1

Formulate a linear program for this capacity expansion problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations).

Problem 3.2

How much should the utility build of each type of generating plant? What will the total cost be? How much energy will be non-served?

Problem 3.3

What fraction of annual generation does each plant type produce? How does this compare to the breakdown of built capacity that you found in Problem 3.2? Do these results make sense given the demand and generator data?

Problem 3.4

Make a plot of the electricity price in each hour. Discuss any trends that you see.

Problem 3.5

What is the shadow price of CO2? What is the value to the utility be of allowing it to emit an additional 1000 tCO2/yr?

Significant Digits

Use `round(x; digits=n)` to report values to the appropriate precision! If your number is on a different order of magnitude and you want to round to a certain number of significant digits, you can use `round(x; sigdigits=n)`.

Getting Variable Output Values

`value.(x)` will report the values of a `JuMP` variable `x`, but it will return a special container which holds other information about `x` that is useful for `JuMP`. This means that you can't use this output directly for further calculations. To just extract the values, use `value.(x).data`.

Suppressing Model Command Output

The output of specifying model components (variable or constraints) can be quite large for this problem because of the number of time periods. If you end a cell with an `@variable` or `@constraint` command, I *highly* recommend suppressing output by adding a semi-colon after the last command, or you might find that your notebook crashes.

```
# Problem 3.1: Formulate the capacity expansion LP (build model)

# Load data
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
rename!(NY_demand, :"Time Stamp" => :Date)
demand_df = NY_demand[:, [:Date, :C]]
rename!(demand_df, :C => :Demand)
demand_df.Hour = 1:nrow(demand_df)
demand = demand_df.Demand                # vector of hourly demand (MWh)
n_hours = length(demand)

gens = DataFrame(CSV.File("data/generators.csv"))
plants = Vector{String}(gens.Plant)
fixed_cost = Dict(gens.Plant .=> Float64.(gens.FixedCost))    # $/MW (annualized in the model
var_cost   = Dict(gens.Plant .=> Float64.(gens.VarCost))      # $/MWh
emission   = Dict(gens.Plant .=> Float64.(gens.Emissions))    # tCO2 / MWh

cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))
wind_cf = cap_factor.Wind
solar_cf = cap_factor.Solar

penalty_ns = 10_000.0    # $/MWh for non-served energy
co2_limit = 1.5e6        # 1.5 MtCO2 per year

# Build JuMP model
model_31 = Model(HiGHS.Optimizer)
set_silent(model_31)   # suppress solver output for readability

# Decision variables
@variable(model_31, cap[plants] >= 0)               # capacity in MW for each plant type
@variable(model_31, gen[plants, 1:n_hours] >= 0)   # hourly generation in MWh
@variable(model_31, ns[1:n_hours] >= 0)            # non-served energy (MWh)

# Hourly supply constraint (allowing ns as slack variable)
for t in 1:n_hours
    @constraint(model_31, sum(gen[p,t] for p in plants) + ns[t] >= demand[t])
end

# Generation availability constraints (plant-specific)
for p in plants
    if p == "Geothermal"
        # geothermal limited to 85% of installed capacity at any hour
```

```
        for t in 1:n_hours
            @constraint(model_31, gen[p,t] <= 0.85 * cap[p])
        end
    elseif p == "Wind"
        for t in 1:n_hours
            @constraint(model_31, gen[p,t] <= wind_cf[t] * cap[p])
        end
    elseif p == "Solar"
        for t in 1:n_hours
            @constraint(model_31, gen[p,t] <= solar_cf[t] * cap[p])
        end
    else
        # Coal, NG CCGT, NG CT can use full capacity
        for t in 1:n_hours
            @constraint(model_31, gen[p,t] <= cap[p])
        end
    end
end

# Annual CO2 constraint
co2_constraint_31 = @constraint(model_31, sum(gen[p,t] * emission[p] for p in plants, t in 1

# Objective: minimize fixed + variable + penalty costs
@expression(model_31, total_fixed, sum(fixed_cost[p] * cap[p] for p in plants))
@expression(model_31, total_var,   sum(var_cost[p] * gen[p,t] for p in plants, t in 1:n_hours
@expression(model_31, total_ns,    penalty_ns * sum(ns[t] for t in 1:n_hours))

@objective(model_31, Min, total_fixed + total_var + total_ns)

# Solve
optimize!(model_31)
```

PROBLEM 3.1

The decision variables represent the installed capacity of each plant type, the hourly generation from each plant, and the amount of unmet (non-served) demand. The objective function minimizes the sum of annual fixed costs, hourly variable generation costs, and penalties for unserved energy. The constraints include an hourly energy balance ensuring that generation plus any unmet energy equals or exceeds demand, generation limits based on plant-specific capacity factors, and an annual CO emissions cap. This model is linear because all decision variables, cost terms, and constraints are expressed as linear relationships. Including a penalty for non-served energy prevents infeasibility by allowing limited unmet demand at a

high economic cost. Overall, the linear program jointly determines both capacity investment and operational dispatch in a way that reflects economic trade-offs among cost, reliability, and environmental compliance.

```
# Problem 3.2: Extract and report capacities, costs, non-served energy

status = termination_status(model_31)
if status != MOI.OPTIMAL
    @warn "Model did not terminate optimally: $status"
end

# Extract capacities
cap_vals = Dict(p => value(cap[p]) for p in plants)

println("\n--- Built capacities (MW) ---")
for p in plants
    @printf("  %-10s : %8.3f MW\n", p, cap_vals[p])
end

# Cost components
fixed_val = value(total_fixed)
var_val   = value(total_var)
ns_val    = value(total_ns)
total_cost = objective_value(model_31)

@printf("\nTotal cost components (annual):\n  fixed = %.2f\n  variable = %.2f\n  non-served p
        fixed_val, var_val, ns_val, total_cost)

# Total non-served energy
nons_total = sum(value(ns[t]) for t in 1:n_hours)
@printf("\nTotal non-served energy = %.3f MWh per year\n", nons_total)

# Annual generation by plant
gen_totals = Dict(p => sum(value(gen[p,t]) for t in 1:n_hours) for p in plants)
total_generation = sum(values(gen_totals))

println("\n--- Annual generation (MWh) and fraction of total generation ---")
for p in plants
    frac = total_generation > 0 ? gen_totals[p] / total_generation : 0.0
    @printf("  %-10s : %10.1f MWh   (%6.2f%%)\n", p, gen_totals[p], 100*frac)
end
```

```
--- Built capacities (MW) ---
  Geothermal :  285.568 MW
  Coal       :    0.000 MW
  NG CCGT    : 1509.173 MW
  NG CT      :  703.062 MW
  Wind       : 2548.888 MW
  Solar      : 2103.749 MW

Total cost components (annual):
  fixed = 677168118.34
  variable = 104861933.47
  non-served penalty = 3322907.14
  TOTAL = 785352958.94

Total non-served energy = 332.291 MWh per year

--- Annual generation (MWh) and fraction of total generation ---
  Geothermal :  1109986.2 MWh   (  6.78%)
  Coal       :        0.0 MWh   (  0.00%)
  NG CCGT    :  3322766.6 MWh   ( 20.30%)
  NG CT      :   129473.4 MWh   (  0.79%)
  Wind       :  6607522.3 MWh   ( 40.37%)
  Solar      :  5197825.1 MWh   ( 31.76%)
```

PROBLEM 3.2

After solving the optimization model, the results provide a detailed breakdown of installed capacities, annual generation, total cost, and non-served energy. The installed capacities show how much of each technology the optimizer recommends building, while the annual generation values reflect how those technologies are actually utilized across all hours of the year. The total annual cost is divided into fixed, variable, and penalty components, allowing analysis of whether the system is capital-intensive (dominated by fixed costs) or operation-intensive (dominated by variable costs). Non-served energy measures the total amount of demand not met due to cost or capacity limitations; given the very high penalty ($10,000/MWh), the model will only allow it in extreme circumstances. Technologies with high fixed costs but low variable costs (such as renewables or geothermal) tend to provide steady or base load generation, while those with low fixed costs and high variable costs (like gas turbines) are more suitable for peaking or reserve roles. This interpretation of the results clarifies how the optimization balances cost, availability, and emissions constraints to determine the most economical and feasible generation mix.

```
# Problem 3.3: Compare generation fraction to installed capacity fraction

# Capacity fractions
cap_total = sum(cap_vals[p] for p in plants)
println("\n--- Installed capacity (MW) and fraction ---")
for p in plants
    capfrac = cap_total > 0 ? cap_vals[p] / cap_total : 0.0
    @printf("  %-10s : %8.3f MW   (%6.2f%%)\n", p, cap_vals[p], 100*capfrac)
end

# Re-print generation fractions for clarity (already computed)
println("\n--- Reiterating generation fractions ---")
for p in plants
    genfrac = total_generation > 0 ? gen_totals[p] / total_generation : 0.0
    @printf("  %-10s : %10.1f MWh   (%6.2f%%)\n", p, gen_totals[p], 100*genfrac)
end

# This explicitly compares:
#  - the share of installed capacity each technology represents (MW fraction), and
#  - the share of annual generation each technology provides (MWh fraction).
#
# Why this comparison matters:
#  - Capacity fraction alone does not capture utilization. A wind farm with low average
#    capacity factor might occupy a small or moderate share of installed capacity but
#    produce proportionally less energy, or sometimes more (if capacity factor is high).
#  - Thermal plants (coal, gas) are dispatchable and can be used when needed; hence they
#    may show smaller capacity but provide significant peak generation, or conversely,
#    larger capacity but modest generation if used as reserve.
#  - Renewable resources (wind/solar) often require much less scheduled capacity per MWh
#    produced if their capacity factors are high at times of demand; yet they are variable ar
#    their hourly availability shapes how much firm capacity needs to be installed around the
#
# Do results make sense?
#  - If a low-capacity wind build produces a relatively large share of energy, that implies
#    strong wind capacity factors during demand hours.
#  - If gas turbines show large capacity but low generation, they may be retained for reliabi
#    (peaking) rather than base load due to high variable costs.
#  - Compare the generation mix with the cost and emission parameters: if CO2 is constrained
#    the model will favor low-emission tech (geothermal, wind, solar) even if they have highe
#    fixed costs, up to the point where marginal cost or availability reduces their competiti
#
# The comparison therefore provides insight into both economic drivers (costs) and physical
```

```
# drivers (capacity factors and CO2 limits) that determine the optimal portfolio.
```

```
--- Installed capacity (MW) and fraction ---
  Geothermal :  285.568 MW   (  3.99%)
  Coal       :    0.000 MW   (  0.00%)
  NG CCGT    : 1509.173 MW   ( 21.11%)
  NG CT      :  703.062 MW   (  9.83%)
  Wind       : 2548.888 MW   ( 35.65%)
  Solar      : 2103.749 MW   ( 29.42%)

--- Reiterating generation fractions ---
  Geothermal :  1109986.2 MWh   (  6.78%)
  Coal       :        0.0 MWh   (  0.00%)
  NG CCGT    :  3322766.6 MWh   ( 20.30%)
  NG CT      :   129473.4 MWh   (  0.79%)
  Wind       :  6607522.3 MWh   ( 40.37%)
  Solar      :  5197825.1 MWh   ( 31.76%)
```

```
# Problem 3.5: Shadow price of CO2 (dual) and value of +1000 tCO2

# Rebuild the model but retain a handle to the CO2 constraint so we can query its dual.
model_35 = Model(HiGHS.Optimizer)
set_silent(model_35)

@variable(model_35, capb[plants] >= 0)
@variable(model_35, genb[plants, 1:n_hours] >= 0)
@variable(model_35, nsb[1:n_hours] >= 0)

for t in 1:n_hours
    @constraint(model_35, sum(genb[p,t] for p in plants) + nsb[t] >= demand[t])
end

for p in plants
    if p == "Geothermal"
        for t in 1:n_hours
            @constraint(model_35, genb[p,t] <= 0.85 * capb[p])
        end
    elseif p == "Wind"
        for t in 1:n_hours
            @constraint(model_35, genb[p,t] <= wind_cf[t] * capb[p])
```

```
        end
    elseif p == "Solar"
        for t in 1:n_hours
            @constraint(model_35, genb[p,t] <= solar_cf[t] * capb[p])
        end
    else
        for t in 1:n_hours
            @constraint(model_35, genb[p,t] <= capb[p])
        end
    end
end

co2_con_handle = @constraint(model_35, sum(genb[p,t] * emission[p] for p in plants, t in 1:n

@objective(model_35, Min,
    sum(fixed_cost[p]*capb[p] for p in plants) +
    sum(var_cost[p]*genb[p,t] for p in plants, t in 1:n_hours) +
    penalty_ns * sum(nsb[t] for t in 1:n_hours)
)

optimize!(model_35)
status35 = termination_status(model_35)
if status35 != MOI.OPTIMAL
    @warn "Model for shadow price did not terminate optimally: $status35"
end

shadow_co2 = dual(co2_con_handle)   # $ per tCO2 (since objective is in $)
println("Shadow price of CO2 constraint: ", round(shadow_co2, digits=2), " per tCO2")
println("Value of allowing +1000 tCO2/yr: ", round(1000 * shadow_co2, digits=2))
```

```
Shadow price of CO2 constraint: -182.77 per tCO2
Value of allowing +1000 tCO2/yr: -182771.94
```

PROBLEM 3.5

The shadow price of the CO constraint represents the marginal cost to the system of one additional tonne of CO emissions. In other words, it quantifies how much total system cost would decrease if the emissions cap were relaxed by a single tonne. A positive shadow price indicates that the CO limit is binding—allowing more emissions would enable the use of cheaper but dirtier generation sources and therefore reduce costs. A zero shadow price implies that the constraint is not active, meaning the system already operates below the cap and relaxing it would have no effect. The product of the shadow price and 1,000 gives the total

value to the utility of being allowed to emit an additional 1,000 tonnes of CO per year. This information has real-world implications for carbon policy and market design: it helps determine whether purchasing carbon credits or investing in emissions abatement is more cost-effective. Because shadow prices reflect marginal (small) changes, their interpretation is most accurate for modest adjustments to the constraint. Overall, this analysis provides a quantitative measure of the economic impact of environmental regulation on system operation and investment decisions.

References

List any external references consulted, including classmates.